

D. Y. Patil College of Engineering & Technology Kolhapur

Department of Computer Science & Engineering

Course Name: Object Oriented Concepts

Class – Second Year B. Tech

Course Code: 231CSELPCCP204

Sem –III

Experiment No.: 2

Title: Implement a program to create a Complex class to represent complex numbers. Implement member functions for addition, subtraction, and display of complex numbers

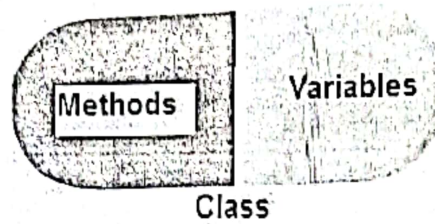
Theory: In this section we will see how to create and use complex numbers in C++. We can create a complex number class in C++, that can hold the real and imaginary part of the complex number as member elements. There will be some member functions that are used to handle this class.

In this example we are creating one complex type class, a function to display the complex number into the correct format.

Encapsulation

In normal terms, Encapsulation is defined as wrapping up data and information under a single unit. In Object-Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them. Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section, etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name "sales section".

Encapsulation in C++



Encapsulation in C++

Encapsulation also leads to data abstraction or data hiding. Using encapsulation also hides the data. In the above example, the data of any of the sections like sales, finance, or accounts are hidden from any other section.

// C++ program to Add two complex numbers

```
#include <bits/stdc++.h>

using namespace std;

class Complex {
public:
    int real; // To store real part of complex number
    int imaginary; // To store imaginary part of complex number
    Complex()
    {
        // Initial values are zero
        real = 0;
        imaginary = 0;
    }
    Complex(int r, int i)
    {
        real = r; // r is initialized during object creation
        imaginary = i; // i is initialized during object creation
    }
}
```

```

    }
    Complex addComplexNumber(Complex C1, Complex C2)
    {
        Complex res; // result object of complex class
        // adding real part of complex numbers
        res.real = C1.real + C2.real;
        // adding Imaginary part of complex numbers
        res.imaginary = C1.imaginary + C2.imaginary;
        // returning the sum
        return res;
    }
};

// Main Class
int main()
{
    // First Complex number
    Complex C1(4, 5);
    // printing first complex number
    cout << "Complex number 1 : " << C1.real
        << " + i" << C1.imaginary << endl;

    // Second Complex number
    Complex C2(8, 9);
    // printing second complex number
    cout << "Complex number 2 : " << C2.real
        << " + i" << C2.imaginary << endl;

    // for Storing the sum
    Complex C3;
    // calling addComplexNumber() method

```

```

C3 = C3.addComplexNumber(C1, C2);
// printing the sum
cout << "Sum of complex number : "
    << C3.real << " + i"
    << C3.imaginary;
cout << endl
    << endl;

// Test for second input
// First Complex number
Complex A(2, 7);
// printing first complex number
cout << "Complex number 1 : " << A.real
    << " + i" << A.imaginary << endl;
// Second Complex number
Complex B(10, 6);
// printing second complex number
cout << "Complex number 2 : " << B.real
    << " + i" << B.imaginary << endl;
// for Storing the sum
Complex C;
// calling addComplexNumber() method
C = C.addComplexNumber(A, B);
// printing the sum
cout << "Sum of complex number : "
    << C.real << " + i"
    << C.imaginary;
}

```

Output:

Complex number 1 : $4 + i5$

Complex number 2 : $8 + i9$

Sum of complex number : $12 + i14$

Complex number 1 : $2 + i7$

Complex number 2 : $10 + i6$

Sum of complex number : $12 + i13$

Results –

1. Students are able to understand the use of Class.

Observations –

1. Students are able to use the class at appropriate real time problems.

Prepared By:

Mrs. S. V. Shinge, Mrs. R. S. Jadhav, Mrs. S. G. Ambi